MQL5 on social media
Follow us
Don't miss updates
>>



Построение системы обнаружения дивергенций (Часть IV): Создание переиспользуемого модуля обнаружения дивергенций для MQL5

0 Solomon Anietie Sunday 54

Содержание

1. Введение
2. Зачем проводить рефакторинг?
3. Проектирование подключаемого файла
   - Этап 1 — основа
   - Этап 2 — модуль осциллятора
   - Этап 3 — модуль обнаружения дивергенций
   - Этап 4 — функции доступа
4. Контрольный список для подключения модуля
5. Выбор и адаптация индикатора Parabolic SAR
6. Создание демонстрационного советника
7. Результаты
8. Заключение

Введение

В первых трех частях этой серии мы с нуля создали полноценную адаптивную торговую систему. В первой части серии был представлен осциллятор MPO4 — показатель моментума, рассчитываемый на основе взвешенных тел свечей. Во второй части этот осциллятор или, в качестве альтернативы, RSI использовался для обнаружения дивергенций и адаптации индикатора SuperTrend. Множитель ATR уменьшался всякий раз, когда дивергенция против текущего тренда сигнализировала о его ослаблении. В третьей части та же адаптивная логика была перенесена в полностью автономного советника и дополнена расчетом размера позиции, стоп-лоссами на основе ATR, логикой трейлинга и сессионными фильтрами.

Проблема повторного использования

По мере развития серии статей одну деталь становилось все труднее игнорировать. Модуль обнаружения дивергенций — расчет значений осциллятора, обнаружение опорных точек, классификация дивергенций и хранение состояния — до сих пор существовал только внутри реализации SuperTrend. Код был написан один раз — для одного индикатора. Чтобы повторно использовать его в другом проекте, пришлось бы скопировать туда тот же блок кода и надеяться, что при переносе ничего не сломается. Это уже не архитектурное решение, а источник проблем с сопровождением.

Решение

В этой статье мы устраним это ограничение. Мы вынесем модуль обнаружения дивергенций в отдельный автономный подключаемый файл DivergenceEngine.mqh. Его можно будет подключать к

различным индикаторам и советникам и инициализировать модуль независимо от какого-либо конкретного индикатора. Не менее важно, что мы сосредоточимся на подключении этого файла к вашему собственному проекту, а не только к демонстрационному примеру из этой статьи.

Демонстрационный пример намеренно прост: это небольшой советник на основе **Parabolic SAR**, в котором **коэффициент ускорения** индикатора изменяется при появлении дивергенции против текущего тренда. Цель — показать работу модуля обнаружения дивергенций с минимальным объемом сопутствующего кода. Основное внимание в статье уделяется схеме подключения модуля, а не использующей его торговой логике.

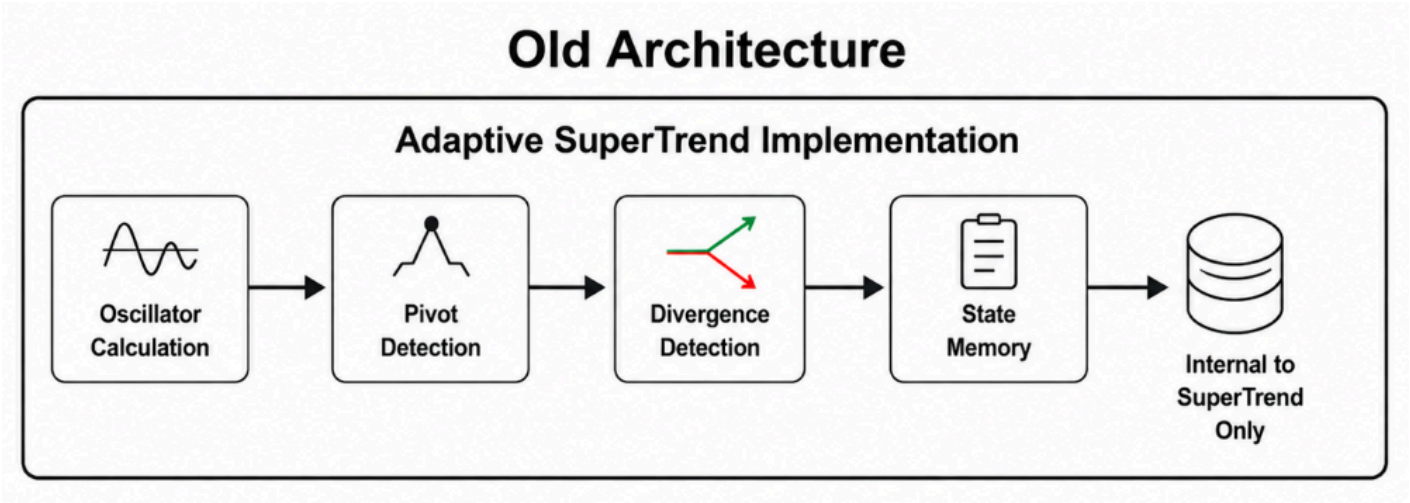
Что рассматривается в этой статье

В следующих разделах мы:

- рассмотрим, почему текущая архитектура затрудняет повторное использование кода, и опишем новую архитектуру;
- спроектируем DivergenceEngine.mqh в четыре этапа — основа, модуль осциллятора, модуль обнаружения дивергенций и функции доступа;
- разберем общий контрольный список для подключения файла к любому вашему индикатору или советнику;
- используем Parabolic SAR для демонстрации и разберем ограничение работы со встроенным индикатором через хэндл на языке MQL5, которое определяет способ реализации адаптивной логики;
- создадим небольшой советник Adaptive Parabolic SAR с использованием подключаемого файла;
- сравним в тестах работу стандартного и адаптивного вариантов и проанализируем полученные результаты.

Зачем проводить рефакторинг?

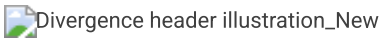
В первых трех частях в Adaptive SuperTrend было реализовано все необходимое для обнаружения дивергенций: расчет значений осциллятора (MPO4 или RSI), обнаружение опорных точек, классификация дивергенций и управление состоянием. В этом состоянии хранятся цена последней опорной точки, последнее значение осциллятора, тип последней дивергенции и количество баров, прошедших с момента ее появления. Каждый компонент добавлялся там, где требовался для расчета SuperTrend, с использованием тех же массивов с индексацией типа "временные ряды", которые используются в индикаторе.
Это разумный подход для одной автономной системы, но у него есть недостаток: ни один из этих компонентов не существует независимо от остальных. Они встроены в цикл расчета SuperTrend, размеры используемых массивов зависят от его периода анализа, а при инициализации применяются граничные значения, характерные для SuperTrend. Если другому индикатору или не связанному с SuperTrend советнику требуется такая же логика обнаружения дивергенций, отдельного интерфейса для доступа к ней нет. На практике остается скопировать соответствующий код в новый проект и дорабатывать его до тех пор, пока он не скомпилируется. Такое "повторное использование" становится проблемой сопровождения кода, когда в одной копии исправляют ошибку, а в другой — нет. Это схематично показано ниже:



Осциллятор, обнаружение опорных точек, обнаружение дивергенций и хранение состояния объединены в одной реализации и могут использоваться только в ней.

Новая архитектура — DivergenceEngine.mqh

Решение — вынести логику обнаружения дивергенций в отдельный подключаемый файл, не зависящий от SuperTrend, множителей ATR или трендовых полос. Модуль в файле DivergenceEngine.mqh принимает ценовые данные и формирует состояние дивергенции. Основные функции остаются прежними: расчет значений осциллятора, обнаружение опорных точек, обнаружение дивергенций и управление состоянием. Разница в том, что эти компоненты объединены в единой структуре, объявленной с помощью struct, а не распределены по глобальным массивам, привязанным к одному проекту. Любой индикатор или советник, которому нужно учитывать дивергенции, может подключить этот файл, создать собственный экземпляр DivergenceEngine и независимо получать из него данные. Как использовать результат, полностью определяет вызывающий код: сузить трендовую полосу, скорректировать коэффициент ускорения, разрешить или запретить вход в позицию либо использовать результат в каком-либо будущем адаптивном механизме. У модуля одна задача: определить, есть ли дивергенция и сколько времени прошло с момента ее появления. Это схематично показано ниже:



Модуль в файле DivergenceEngine.mqh независим от использующих его программ: индикатора Adaptive SuperTrend, советника Adaptive SuperTrend, советника Adaptive Parabolic SAR, который будет создан далее в этой статье, и любых будущих проектов, в которых потребуется обнаружение дивергенций.

Преимущества рефакторинга

Выделение логики обнаружения дивергенций в отдельный модуль дает ряд практических преимуществ:

- Отсутствие дублирования кода. Логика обнаружения дивергенций реализуется один раз и используется всеми индикаторами и советниками, которые подключают этот файл.
- Более простое сопровождение. Исправления ошибок и улучшения в файле DivergenceEngine.mqh распространяются на все проекты, в которых он используется.
- Независимое тестирование. Модуль можно проверять отдельно от любой торговой стратегии, которая его использует.
- Более широкие возможности повторного использования. Любой индикатор или советник, способный передавать необходимые ценовые данные, может использовать этот модуль независимо от того, как в нем обрабатывается обнаруженная дивергенция.
- Более упорядоченная структура проектов. В индикаторах и советниках реализуются только их собственные задачи, без повторной реализации расчета значений осциллятора и обнаружения опорных точек.

Проектирование подключаемого файла

Теперь, когда необходимость рефакторинга обоснована, мы можем приступить к созданию файла DivergenceEngine.mqh. Создание подключаемого файла разделено на четыре этапа — основа, модуль осциллятора, модуль обнаружения дивергенций и функции доступа. На каждом этапе реализуется отдельная часть задач модуля. Вместе эти части образуют полноценный модуль.

Начнем с создания в MetaEditor нового файла .mqh, в котором будут указаны только базовые свойства:

```
//+------------------------------------------------------------------+
//|                                              DivergenceEngine.mqh |
//|                                                 Utility / Library |
//|                                  Copyright 2026, soloharbinger |
//|                       https://www.mql5.com/ru/users/soloharbinger |
//+------------------------------------------------------------------+
#property copyright "Copyright 2026, soloharbinger"
#property link      "https://www.mql5.com/ru/users/soloharbinger"
```

Этап 1 — основа

На этапе создания основы определяются данные, с которыми работает остальной код подключаемого файла, а также функции инициализации, освобождения ресурсов и управления буферами. Все начинается с выбора источника, структуры DivergenceEngine и служебных значений, задающих нейтральное состояние по умолчанию:

```
//+------------------------------------------------------------------+
//| Phase 1/4 - FOUNDATION                                           |
//+------------------------------------------------------------------+

//--- Sentinel values for "no data"
#define DIV_NO_BARS         INT_MAX/4   // e.g., 536870911
#define DIV_NO_LOW_PRICE    DBL_MAX
#define DIV_NO_HIGH_PRICE   -DBL_MAX
#define DIV_NO_LOW_OSC      DBL_MAX
#define DIV_NO_HIGH_OSC     -DBL_MAX

//--- Oscillator source
enum ENUM_DIV_SOURCE
  {
   DIV_SRC_MPO = 0,     // MPO4 Oscillator
   DIV_SRC_RSI = 1      // Standard RSI
  };

//--- Global Structure
struct DivergenceEngine
  {
   //--- Settings
   ENUM_DIV_SOURCE   Source;
   int               MPO_len;
   int               MPO_smooth;
   int               RsiPeriod;
   int               DivPivotLen;
   double            SmoothAlpha;

   //--- Handles
   int               rsiHandle;

   //--- Hidden Calculation Buffers (Standard dynamic arrays)
   double            buf_OscRaw[];
```

```
   double            buf_OscSmooth[];
   double            buf_RSI[];

   //--- Hidden State Memory Buffers
   double            LastPivLowPriceBuffer[];
   double            LastPivLowOscBuffer[];
   double            LastPivHighPriceBuffer[];
   double            LastPivHighOscBuffer[];

   int               LastTypeBuffer[];       // 1=Bull, -1=Bear, 0=None
   int               BarsSinceBuffer[];      // Tracks bars since last divergence
  };
```

Все, что необходимо модулю, — настройки, хэндл RSI при необходимости, расчетные буферы и буферы состояния — теперь хранится в единой структуре. В коде, использующем модуль, можно объявить один экземпляр DivergenceEngine вместо управления набором разрозненных глобальных массивов. Кроме того, несколько экземпляров могут работать одновременно, не мешая друг другу: например, один может использовать MPO4, а другой — RSI.

Для инициализации предусмотрена специальная вспомогательная функция, поэтому в вызывающем коде не нужно настраивать структуру вручную:

```
//+------------------------------------------------------------------+
//| Initialization helper function                                   |
//+------------------------------------------------------------------+
bool InitializeDivergenceEngine(DivergenceEngine &divEngine,
                                ENUM_DIV_SOURCE source,
                                int mpoLen,
                                int mpoSmooth,
                                int rsiPeriod,
                                int divPivotLen,
                                string symbol = "",
                                ENUM_TIMEFRAMES tf = PERIOD_CURRENT)
  {
   if(mpoLen <= 0 || mpoSmooth <= 0 || rsiPeriod <= 0 || divPivotLen <= 0)
     {
      Print("Invalid DivergenceEngine parameters: all must be > 0");
      return false;
     }
   if(mpoSmooth < mpoLen)
      Print("Caution: MPO smooth less than MPO length may cause short smoothing");

   if(symbol == "")
      symbol = _Symbol;

   divEngine.Source = source;
   divEngine.MPO_len = mpoLen;
   divEngine.MPO_smooth = mpoSmooth;
   divEngine.RsiPeriod = rsiPeriod;
   divEngine.DivPivotLen = divPivotLen;
   divEngine.SmoothAlpha = 2.0 / (mpoSmooth + 1.0);
   divEngine.rsiHandle = INVALID_HANDLE;

   if(divEngine.Source == DIV_SRC_RSI)
     {
      divEngine.rsiHandle = iRSI(symbol, tf, divEngine.RsiPeriod, PRICE_CLOSE);
      if(divEngine.rsiHandle == INVALID_HANDLE)
```

```
         {
          Print("Failed to create RSI handle in DivergenceEngine");
          return false;
         }
      }
    return true;
   }
```

Хэндл индикатора RSI создается только при выборе DIV_SRC_RSI. Для MPO4 не требуется хэндл индикатора, поскольку его значения рассчитываются непосредственно на основе ценовых данных. Аналогичным образом выполняется деинициализация:

```
//+------------------------------------------------------------------+
//| Array release helper function                                    |
//+------------------------------------------------------------------+
void ReleaseDivergenceEngine(DivergenceEngine &divEngine)
  {
   if(divEngine.rsiHandle != INVALID_HANDLE)
     {
      if(!IndicatorRelease(divEngine.rsiHandle))
         PrintFormat("IndicatorRelease failed: handle=%d, error=%d",
                     divEngine.rsiHandle, GetLastError());
      divEngine.rsiHandle = INVALID_HANDLE;
     }

   ArrayFree(divEngine.buf_OscRaw);
   ArrayFree(divEngine.buf_OscSmooth);
   ArrayFree(divEngine.buf_RSI);
   ArrayFree(divEngine.LastPivLowPriceBuffer);
   ArrayFree(divEngine.LastPivLowOscBuffer);
   ArrayFree(divEngine.LastPivHighPriceBuffer);
   ArrayFree(divEngine.LastPivHighOscBuffer);
   ArrayFree(divEngine.LastTypeBuffer);
   ArrayFree(divEngine.BarsSinceBuffer);
  }
```

Функцию ReleaseDivergenceEngine() следует вызывать в обработчике OnDeinit() программы, использующей модуль, — так же, как при освобождении любого другого хэндла индикатора.

Наконец, на этапе создания основы реализуется управление жизненным циклом буферов. Задаются размеры массивов и начальные значения для бара, из которого при обработке самого старого бара берется состояние, чтобы рекурсивные вычисления начинались с корректного состояния.

```
//+------------------------------------------------------------------+
//| Buffer sizing & mapping function                                 |
//+------------------------------------------------------------------+
bool ResizeDivergenceBuffers(DivergenceEngine &divEngine, int size)
  {
   if(size <= 0)
     {
      PrintFormat("ResizeDivergenceBuffers: invalid size=%d", size);
      return false;
     }

   if(ArrayResize(divEngine.buf_OscRaw, size) != size ||
      ArrayResize(divEngine.buf_OscSmooth, size) != size ||
      ArrayResize(divEngine.buf_RSI, size) != size ||
```

```
      ArrayResize(divEngine.LastPivLowPriceBuffer, size) != size ||
      ArrayResize(divEngine.LastPivLowOscBuffer, size) != size ||
      ArrayResize(divEngine.LastPivHighPriceBuffer, size) != size ||
      ArrayResize(divEngine.LastPivHighOscBuffer, size) != size ||
      ArrayResize(divEngine.LastTypeBuffer, size) != size ||
      ArrayResize(divEngine.BarsSinceBuffer, size) != size)
     {
      PrintFormat("ResizeDivergenceBuffers: ArrayResize failed, size=%d, error=%d",
                  size, GetLastError());
      return false;
     }

   if(!ArraySetAsSeries(divEngine.buf_OscRaw, true) ||
      !ArraySetAsSeries(divEngine.buf_OscSmooth, true) ||
      !ArraySetAsSeries(divEngine.buf_RSI, true) ||
      !ArraySetAsSeries(divEngine.LastPivLowPriceBuffer, true) ||
      !ArraySetAsSeries(divEngine.LastPivLowOscBuffer, true) ||
      !ArraySetAsSeries(divEngine.LastPivHighPriceBuffer, true) ||
      !ArraySetAsSeries(divEngine.LastPivHighOscBuffer, true) ||
      !ArraySetAsSeries(divEngine.LastTypeBuffer, true) ||
      !ArraySetAsSeries(divEngine.BarsSinceBuffer, true))
     {
      PrintFormat("ResizeDivergenceBuffers: ArraySetAsSeries failed, error=%d",
                  GetLastError());
      return false;
     }

   return true;
  }

//+------------------------------------------------------------------+
//| Boundary Initialization function (Seed the bar at limit+1)       |
//+------------------------------------------------------------------+
void InitDivergenceBoundaries(DivergenceEngine &divEngine, int limit)
  {
   int size = ArraySize(divEngine.buf_OscSmooth);
   if(limit + 1 >= size)
      return;

//--- The loop's first bar (i = limit) reads i+1, so limit+1 is the only bar to seed
   divEngine.buf_OscRaw[limit+1] = 0;
   divEngine.buf_OscSmooth[limit+1] = 0;
   divEngine.LastTypeBuffer[limit+1] = 0;
   divEngine.BarsSinceBuffer[limit+1] = DIV_NO_BARS;
//--- Any real price passes the price sentinels and no oscillator value passes the
//--- oscillator ones, so the first pivot becomes a reference, never a divergence
   divEngine.LastPivLowPriceBuffer[limit+1] = DIV_NO_LOW_PRICE;
   divEngine.LastPivLowOscBuffer[limit+1] = DIV_NO_LOW_OSC;
   divEngine.LastPivHighPriceBuffer[limit+1] = DIV_NO_HIGH_PRICE;
   divEngine.LastPivHighOscBuffer[limit+1] = DIV_NO_HIGH_OSC;
  }
```

Функция InitDivergenceBoundaries() вызывается на каждом проходе перед циклом обработки баров. Этот цикл начинается с i = limit, а обе функции, CalculateOscillator() и DetectDivergence(), считывают данные по индексу i + 1. Поэтому при обработке первого бара используются данные по индексу limit + 1. Функция осуществляет запись только по этому индексу, заполняя соответствующие элементы массивов служебными значениями: отсутствием дивергенции, количеством баров DIV_NO_BARS и значениями-

заполнителями для опорных точек. При сравнении с этими значениями-заполнителями любая фактическая цена удовлетворяет ценовому условию проверки дивергенции, но ни одно фактическое значение осциллятора не удовлетворяет условию для осциллятора. Поэтому опорная точка, сравниваемая с ними, лишь сохраняется для сравнения со следующей опорной точкой. Здесь используется тот же способ задания начальных значений на границе расчетного диапазона, что и в исходной реализации SuperTrend. Разница в том, что теперь эта логика реализована в модуле и не дублируется в каждой использующей его программе.

Этап 2 — модуль осциллятора

На этапе создания модуля осциллятора исходные ценовые данные преобразуются в сглаженные значения осциллятора, используемые для обнаружения дивергенций. При выборе RSI функция UpdateDivergenceRSI() копирует значения из буфера индикатора RSI во внутренний массив модуля перед началом основного цикла вычислений:

```
//+------------------------------------------------------------------+
//| Phase 2/4 - OSCILLATOR                                           |
//+------------------------------------------------------------------+
//+------------------------------------------------------------------+
//| Update RSI before calculation loop (If using RSI as source)      |
//+------------------------------------------------------------------+
bool UpdateDivergenceRSI(DivergenceEngine &divEngine, int limit)
  {
   if(divEngine.Source == DIV_SRC_RSI && divEngine.rsiHandle != INVALID_HANDLE)
     {
      // Ensure we have enough bars calculated
      int barsCalc = BarsCalculated(divEngine.rsiHandle);
      if(barsCalc < limit + 2)
        {
         Print("RSI is not ready: BarsCalculated = ", barsCalc, ", needed = ", limit+2)
         return false;
        }

      double tempRSI[];
      if(!ArraySetAsSeries(tempRSI, true))
        {
         PrintFormat("ArraySetAsSeries(tempRSI) failed, error=%d", GetLastError());
         return false;
        }
      int copied = CopyBuffer(divEngine.rsiHandle, 0, 0, limit + 2, tempRSI);
      if(copied < limit + 2)
        {
         Print("Failed to copy enough RSI values. Copied ", copied, ", needed ", limit+
         return false;
        }

      // Clear the whole target range first
      ArrayFill(divEngine.buf_RSI, 0, limit + 2, 0.0);

      // Copy only up to limit+1
      int max_idx = (int)MathMin(limit + 1, copied - 1);
      for(int k = 0; k <= max_idx; k++)
         divEngine.buf_RSI[k] = tempRSI[k];
      return true;
     }
   return true;
  }
```

Затем функция CalculateOscillator() вызывается для каждого обрабатываемого бара. Задача этой функции — рассчитать исходное значение выбранного осциллятора и применить то же экспоненциальное сглаживание, что и в исходной реализации:

```
//+------------------------------------------------------------------+
//| Compute the smoothed oscillator value for the selected source    |
//+------------------------------------------------------------------+
void CalculateOscillator(DivergenceEngine &divEngine, int i, const double &open[], cons
  {
   int size = (int)MathMin(
                  MathMin(ArraySize(open), ArraySize(close)),
                  MathMin(ArraySize(divEngine.buf_OscRaw), ArraySize(divEngine.buf_OscSm
              );

//--- Guard
   if(i < 0 || i >= size)
      return;

   if(divEngine.Source == DIV_SRC_MPO)
     {
      //--- Check if we have enough bars ahead for MPO
      if(i + divEngine.MPO_len > size)
        {
         // Not enough data: carry forward previous value if possible, else set to 0
         if(i + 1 < size)
           {
            divEngine.buf_OscRaw[i] = divEngine.buf_OscRaw[i+1];
            divEngine.buf_OscSmooth[i] = divEngine.buf_OscSmooth[i+1];
           }
         else
           {
            divEngine.buf_OscRaw[i] = 0;
            divEngine.buf_OscSmooth[i] = 0;
           }
         return;
        }

      double rollingSum = 0;
      double sumBodies = 0;
      for(int k = 0; k < divEngine.MPO_len; k++)
         sumBodies += MathAbs(close[i+k] - open[i+k]);
      double avgBody = (sumBodies > 0) ? sumBodies / divEngine.MPO_len : DivPoint;

      for(int k = 0; k < divEngine.MPO_len; k++)
        {
         int idx = i + k;
         double body = MathAbs(close[idx] - open[idx]);
         double dir = (close[idx] > open[idx]) ? 1.0 : (close[idx] < open[idx]) ? -1.0
         double weight = (avgBody > 0) ? body / avgBody : 1.0;
         rollingSum += (dir * weight);
        }
      divEngine.buf_OscRaw[i] = (rollingSum / (divEngine.MPO_len * 2.0)) * 100.0;
     }
   else // RSI source
     {
      //--- Ensure buf_RSI has data at i
      if(i < ArraySize(divEngine.buf_RSI))
```

```
            divEngine.buf_OscRaw[i] = divEngine.buf_RSI[i];
         else
            divEngine.buf_OscRaw[i] = 0;
        }

   //--- Apply Smoothing
      if(i + 1 < size)
         divEngine.buf_OscSmooth[i] = (divEngine.buf_OscRaw[i] * divEngine.SmoothAlpha) +
      else
         divEngine.buf_OscSmooth[i] = divEngine.buf_OscRaw[i]; // fallback
     }
```

В ветке MPO4 сохраняется расчет по взвешенным телам свечей, представленный в первой части серии: размер тела каждой свечи с учетом ее направления взвешивается относительно среднего размера тела в окне. Полученные значения суммируются по MPO_len барам, а результат масштабируется до диапазона значений осциллятора. Главное дополнение — защита от нехватки данных, к которым расчет обращается далее по индексам. Поскольку модуль обрабатывает бары в обратном порядке, при ограниченном объеме истории для самых старых баров может не хватать данных для полного окна длиной MPO_len баров. Чтобы не получить искаженное значение, при расчете по возможности используется предыдущее значение.

В ветке RSI собственные вычисления не выполняются. В ней считываются данные из буфера buf_RSI, заполненного функцией UpdateDivergenceRSI(). Затем к значениям из обоих источников применяется одно и то же экспоненциальное сглаживание с параметром SmoothAlpha. Таким образом, на третьем этапе используется единый буфер осциллятора независимо от того, выбран ли в качестве источника MPO4 или RSI.

Этап 3 — модуль обнаружения дивергенций

При обработке каждого бара функция DetectDivergence() выполняет две операции: наследует предыдущее состояние и данные об опорных точках, а затем проверяет, подтверждается ли новая опорная точка в текущем окне.

```
//+------------------------------------------------------------------+
//| Phase 3/4 - DIVERGENCE                                           |
//+------------------------------------------------------------------+
void DetectDivergence(DivergenceEngine &divEngine, int i, const double &high[], const d
  {
   int size = (int)MathMin(
                  MathMin(ArraySize(high), ArraySize(low)),
                  MathMin(
                     MathMin(ArraySize(divEngine.LastTypeBuffer), ArraySize(divEngine.Ba
                     MathMin(
                        MathMin(ArraySize(divEngine.LastPivLowPriceBuffer), ArraySize(di
                        MathMin(ArraySize(divEngine.LastPivHighPriceBuffer), ArraySize(d
                     )
                  )
               );
   //--- Guard
      if(i < 0 || i >= size)
         return;
      if(i + 1 < size)
        {
         //--- Inherit state
         divEngine.LastTypeBuffer[i]         = divEngine.LastTypeBuffer[i+1];
         divEngine.BarsSinceBuffer[i]        = divEngine.BarsSinceBuffer[i+1] + 1;
         divEngine.LastPivLowPriceBuffer[i]  = divEngine.LastPivLowPriceBuffer[i+1];
         divEngine.LastPivLowOscBuffer[i]    = divEngine.LastPivLowOscBuffer[i+1];
```

```
      divEngine.LastPivHighPriceBuffer[i] = divEngine.LastPivHighPriceBuffer[i+1];
      divEngine.LastPivHighOscBuffer[i]   = divEngine.LastPivHighOscBuffer[i+1];
     }
   else
     {
      //--- Fallback default if at the absolute edge of the array
      divEngine.LastTypeBuffer[i]         = 0;
      divEngine.BarsSinceBuffer[i]        = DIV_NO_BARS;
      divEngine.LastPivLowPriceBuffer[i]  = DIV_NO_LOW_PRICE;
      divEngine.LastPivLowOscBuffer[i]    = DIV_NO_LOW_OSC;
      divEngine.LastPivHighPriceBuffer[i] = DIV_NO_HIGH_PRICE;
      divEngine.LastPivHighOscBuffer[i]   = DIV_NO_HIGH_OSC;
     }

   int pIdx = i + divEngine.DivPivotLen;
//--- Check that we have enough bars on both sides of the pivot and that pIdx is within
   if(pIdx - divEngine.DivPivotLen >= 0 && pIdx + divEngine.DivPivotLen < size && pIdx
     {
      bool isPivLow = true;
      bool isPivHigh = true;
      for(int k = 1; k <= divEngine.DivPivotLen; k++)
        {
         // Strict pivot. For non-strict (allow equal), use <= and >= accordingly.
         if(!(divEngine.buf_OscSmooth[pIdx] < divEngine.buf_OscSmooth[pIdx+k] && divEng
            isPivLow = false;
         if(!(divEngine.buf_OscSmooth[pIdx] > divEngine.buf_OscSmooth[pIdx+k] && divEng
            isPivHigh = false;
        }

      //--- Process Bullish Pivot
      if(isPivLow)
        {
         if(low[pIdx] < divEngine.LastPivLowPriceBuffer[pIdx] && divEngine.buf_OscSmoot
           {
            divEngine.LastTypeBuffer[i] = 1;
            divEngine.BarsSinceBuffer[i] = 0;
           }
         divEngine.LastPivLowPriceBuffer[i] = low[pIdx];
         divEngine.LastPivLowOscBuffer[i] = divEngine.buf_OscSmooth[pIdx];
        }

      //--- Process Bearish Pivot
      if(isPivHigh)
        {
         if(high[pIdx] > divEngine.LastPivHighPriceBuffer[pIdx] && divEngine.buf_OscSmo
           {
            divEngine.LastTypeBuffer[i] = -1;
            divEngine.BarsSinceBuffer[i] = 0;
           }
         divEngine.LastPivHighPriceBuffer[i] = high[pIdx];
         divEngine.LastPivHighOscBuffer[i] = divEngine.buf_OscSmooth[pIdx];
        }
     }
  }
```

Сама опорная точка находится на DivPivotLen баров раньше текущего обрабатываемого бара, по индексу pIdx. Опорный минимум подтверждается, если значение осциллятора в этой точке ниже всех его

значений на DivPivotLen барах с каждой стороны. Для опорного максимума используется противоположное условие. Это тот же алгоритм обнаружения опорных точек в симметричном окне, который был представлен во второй части, но теперь он работает исключительно с внутренним буфером осциллятора модуля, а не с массивами конкретного проекта.

Тип дивергенции определяется только при подтверждении новой опорной точки. Бычья дивергенция возникает, когда цена формирует более низкий минимум, а осциллятор — более высокий минимум. Медвежья дивергенция — обратная ситуация: цена формирует более высокий максимум, а осциллятор — более низкий максимум. При обнаружении любого из этих условий в LastTypeBuffer[i] записывается тип дивергенции, а элементу BarsSinceBuffer[i] присваивается значение ноль. При обработке последующих баров предыдущее состояние просто переносится, а счетчик баров увеличивается. Именно это позволяет модулю отслеживать, сколько баров прошло с момента последней подтвержденной дивергенции.

Этап 4 — функции доступа

Объем работы на заключительном этапе намеренно невелик. Вместо прямого доступа к внутренним буферам модуль предоставляет небольшой набор функций доступа:

```
//+------------------------------------------------------------------+
//| Phase 4/4 - ACCESS                                               |
//+------------------------------------------------------------------+
int GetLastDivergenceType(const DivergenceEngine &divEngine, int i)
  {
   if(i < 0 || i >= ArraySize(divEngine.LastTypeBuffer))
      return 0;
   return divEngine.LastTypeBuffer[i];
  }

//+------------------------------------------------------------------+
//|                                                                  |
//+------------------------------------------------------------------+
int GetBarsSinceDivergence(const DivergenceEngine &divEngine, int i)
  {
   if(i < 0 || i >= ArraySize(divEngine.BarsSinceBuffer))
      return DIV_NO_BARS;
   return divEngine.BarsSinceBuffer[i];
  }

//+------------------------------------------------------------------+
//|                                                                  |
//+------------------------------------------------------------------+
double GetOscillatorValue(const DivergenceEngine &divEngine, int i)
  {
   if(i < 0 || i >= ArraySize(divEngine.buf_OscSmooth))
      return 0.0;
   return divEngine.buf_OscSmooth[i];
  }

//+------------------------------------------------------------------+
//|                                                                  |
//+------------------------------------------------------------------+
void ResetDivergenceState(DivergenceEngine &divEngine, int i)
  {
   if(i < 0 || i >= ArraySize(divEngine.LastTypeBuffer))
      return;
   divEngine.LastTypeBuffer[i] = 0;
   divEngine.BarsSinceBuffer[i] = DIV_NO_BARS;
```

```
    divEngine.LastPivLowPriceBuffer[i] = DIV_NO_LOW_PRICE;
    divEngine.LastPivLowOscBuffer[i] = DIV_NO_LOW_OSC;
    divEngine.LastPivHighPriceBuffer[i] = DIV_NO_HIGH_PRICE;
    divEngine.LastPivHighOscBuffer[i] = DIV_NO_HIGH_OSC;
   }

//+------------------------------------------------------------------+
//|                                                                  |
//+------------------------------------------------------------------+
int GetRequiredLookback(const DivergenceEngine &divEngine)
  {
// Minimum bars needed
   int required = divEngine.MPO_smooth + 2 * divEngine.DivPivotLen + 5;
   if(divEngine.Source == DIV_SRC_RSI)
      required = MathMax(required, divEngine.RsiPeriod + 2 * divEngine.DivPivotLen + 5);
   else // MPO
      required = MathMax(required, divEngine.MPO_len + 2 * divEngine.DivPivotLen + 5);
   return required;
  }
```

Функция GetLastDivergenceType() возвращает 1 для бычьей дивергенции, -1 для медвежьей дивергенции и 0 при отсутствии дивергенции. Функция GetBarsSinceDivergence() возвращает количество баров, прошедших с момента последней подтвержденной дивергенции. Это позволяет в использующем модуль коде со временем ослаблять адаптивное воздействие, а не применять его неограниченно долго. До регистрации дивергенции, а также после сброса счетчик начинается со значения DIV_NO_BARS и затем увеличивается. При обращении по индексу за пределами буфера возвращается само значение DIV_NO_BARS. Поэтому в обоих случаях проверка давности дивергенции, например *barsSince <= DivBarsLimit*, дает отрицательный результат без дополнительной обработки. Функция GetOscillatorValue() возвращает сглаженное значение осциллятора для графического построения или дополнительной фильтрации, а функция ResetDivergenceState() позволяет в использующем модуль коде сбросить состояние дивергенции после выполнения соответствующих действий, например когда разворот тренда делает прежний сигнал недействительным. Функция GetRequiredLookback() рассчитывает минимальный объем истории, необходимый для выбранного осциллятора и заданной длины окна обнаружения опорных точек. Это позволяет в использующей модуль программе легко проверить или скорректировать период анализа до задания размеров буферов модуля.

Эти пять функций образуют открытый интерфейс модуля. Коду вне подключаемого файла не требуется прямой доступ к LastTypeBuffer, buf_OscSmooth или любому другому внутреннему массиву. Любая программа, использующая модуль, в том числе советник Adaptive Parabolic SAR, который будет создан далее в этой статье, взаимодействует с ним через функции доступа, реализованные на четвертом этапе.

Контрольный список для подключения модуля

Задача модуля в подключаемом файле — формировать состояние дивергенции на основе ценовых данных. Во всех программах, использующих модуль, он подключается по одной и той же схеме с использованием существующих обработчиков OnInit() и OnDeinit(), а также основного расчетного цикла.

Этап	Функции	Место вызова	Назначение
Настройка	InitializeDivergenceEngine()	OnInit()	Сохранить настройки и создать хэндл индикатора RSI, если выбран DIV_SRC_RSI.
Период анализа	GetRequiredLookback()	OnInit()	Перед заданием размеров рабочих буферов определить

Этап	Функции	Место вызова	Назначение
			минимальный объем истории, необходимый для выбранного осциллятора и настроек обнаружения опорных точек.
Задание размеров	ResizeDivergenceBuffers()	В OnInit(), а затем повторно перед проходом всякий раз, когда изменяется количество обрабатываемых баров	Задать размеры внутренних буферов модуля в соответствии с количеством баров, обрабатываемых использующей его программой.
Задание начальных значений при каждом обновлении	InitDivergenceBoundaries()	При каждом проходе перед циклом обработки баров	Инициализировать индекс limit + 1, который считывает первый обрабатываемый бар, нейтральными значениями по умолчанию.
Обновление RSI (если используется)	UpdateDivergenceRSI()	Перед основным циклом по барам	Скопировать значения из буфера RSI во внутренний массив модуля.
Цикл обработки баров	CalculateOscillator(), DetectDivergence()	Внутри основного цикла переменная i уменьшается от limit до 0.	Рассчитать значение осциллятора и классифицировать дивергенцию для бара i.
Запрос	GetLastDivergenceType(), GetBarsSinceDivergence(), GetOscillatorValue(), ResetDivergenceState()	В том месте кода, где определяется адаптивная реакция	Считать или сбросить состояние дивергенции.
Завершение работы	ReleaseDivergenceEngine()	OnDeinit()	Освободить хэндл индикатора RSI и буферы.

Последняя деталь: цикл обработки баров должен выполняться в порядке убывания индексов — от самого старого обрабатываемого бара к самому новому. Обе функции, CalculateOscillator() и DetectDivergence(), считывают данные по индексу i + 1, поэтому до обработки текущего бара соответствующие элементы массивов уже должны содержать корректные результаты. Вот так:

```
for(int i = limit; i >= 0; i--)
  {
   CalculateOscillator(divEngine, i, open, close, point);
   DetectDivergence(divEngine, i, high, low, limit);

   // Indicator or EA logic
  }
```

Собственная логика использующей модуль программы — расчет размера позиции, работа с трендовыми полосами, коэффициентами ускорения, фильтрами входа и все остальное — остается вне модуля и определяется самой программой.

Чего не делает модуль

Задача модуля в файле DivergenceEngine.mqh ограничивается определением наличия дивергенции и количества баров, прошедших с момента ее появления. Он не выполняет следующие задачи:

- расчет значений индикатора или выполнение торговой логики использующей модуль программы;
- определение того, какой должна быть адаптивная реакция — уменьшение множителя, изменение параметра, разрешение или запрет входа в позицию и подобные действия определяются в вызывающем коде;
- открытие сделок, управление рисками или построение графических объектов;
- определение того, для чего в конечном итоге будут использоваться его выходные данные.

Это позволяет сохранить разделение ответственности.

Контекст использования в советниках и индикаторах

Последовательность подключения модуля одинакова для индикаторов (OnCalculate, полная история) и советников (OnTick, меньшее локальное окно). Контекст определяет объем доступной истории и частоту выполнения цикла, но сами вызовы функций модуля остаются неизменными.

Выбор и адаптация индикатора Parabolic SAR

Мы установили, что модуль в файле DivergenceEngine.mqh подключается по одной и той же схеме независимо от использующей его программы. Parabolic SAR служит вторым примером, показывающим, что модуль обнаружения дивергенций действительно можно повторно использовать, а не только отделить от SuperTrend. Есть несколько практических причин, по которым он хорошо подходит для демонстрации:

- он доступен как встроенный индикатор на языке MQL5, поэтому многие читатели уже знакомы с ним;
- его чувствительность задается одним входным параметром — коэффициентом ускорения, поэтому в адаптивной логике есть конкретный параметр, который можно изменять;
- работу адаптивного варианта легко визуально сравнить с работой неизмененного варианта, что удобно для проверки результатов.

Дело не в том, что для Parabolic SAR требуется особая схема подключения модуля. Этот индикатор достаточно прост, чтобы основное внимание оставалось на повторном использовании модуля обнаружения дивергенций.

Практические замечания о хэндлах встроенных индикаторов

Прежде чем продолжить, стоит обратить внимание на одно ограничение, связанное с реализацией. В языке MQL5 доступ к встроенному индикатору Parabolic SAR осуществляется через хэндл, который возвращает функция iSAR(). Поэтому встроенный индикатор, доступный через функцию iSAR(), не подходит в качестве источника расчетных значений для этого советника: при обнаружении дивергенции коэффициент ускорения должен изменяться. Поэтому мы будем рассчитывать значения Parabolic SAR непосредственно в советнике по стандартной формуле этого индикатора. Кроме того, для адаптации и графического построения используется один и тот же ряд значений SAR, что упрощает сравнение.

Разделение ответственности

Модуль в файле DivergenceEngine.mqh никогда не рассчитывает значения SAR и никогда не будет этого делать. Код в подключаемом файле никак не учитывает существование Parabolic SAR. Модуль возвращает состояние дивергенции и ее давность через функции доступа, реализованные на четвертом этапе. При необходимости использующая модуль программа также может получить сглаженное значение осциллятора. То, как использовать эту информацию и в какую сторону изменять параметры, полностью определяется в использующей модуль программе.

Советник Adaptive SuperTrend использовал этот сигнал для уменьшения множителя ATR всякий раз, когда дивергенция была направлена против преобладающего тренда. Дивергенция трактовалась как раннее предупреждение о развороте, и линия SuperTrend соответственно приближалась к цене. Советник Adaptive Parabolic SAR использует тот же сигнал, но не ограничивает его трактовку одним вариантом. Дивергенция, направленная против тренда, может указывать на его ослабление и формирование разворота. Однако это может быть и просто откат перед продолжением тренда. Поэтому трейдер может

выбрать, какую трактовку использовать в торговле, задав направление изменения коэффициента ускорения:

```
Adaptive Parameter = Base Parameter * (1.0 - AdaptiveFactor)    // Reduce
Adaptive Parameter = Base Parameter * (1.0 + AdaptiveFactor)    // Increase
```

Уменьшение коэффициента ускорения замедляет приближение SAR к цене после каждого нового максимума или минимума, поэтому SAR дольше остается на большем расстоянии от цены. В этом случае дивергенция трактуется как откат или временная потеря моментума: SAR остается достаточно далеко от цены, чтобы эти колебания не прервали сопровождение текущего тренда. Увеличение коэффициента ускорения дает обратный эффект: SAR быстрее приближается к цене, остается ближе к ней и раньше меняет направление. Это соответствует трактовке дивергенции как раннего предупреждения о развороте, на которое стоит реагировать. Оба варианта являются обоснованными реакциями на один и тот же сигнал. То, какая интерпретация более уместна, зависит от того, как обычно отрабатывают дивергенции на данном рынке. Сам модуль не может определить, какая интерпретация предпочтительнее, поэтому ни одна из них в нем не закреплена.

Модуль в файле DivergenceEngine.mqh предоставляет сигнал и его давность, а в использующей модуль программе определяется, как трактовать этот сигнал и какие действия выполнять на его основе. В следующем разделе показано, как возможность такого выбора реализована в советнике.

Создание демонстрационного советника

Теперь, когда схема подключения модуля определена, мы можем создать демонстрационный советник. Поскольку это демонстрационный пример, а не торговая система для реального использования, сопутствующая торговая логика намеренно упрощена. Основное внимание уделяется инициализации и обновлению модуля в файле DivergenceEngine.mqh, получению данных через его функции доступа и отделению модуля от логики Parabolic SAR.

Входные данные и глобальные переменные

Далее определим точки входа советника и необходимые директивы #property (OnInit, OnTick, OnDeinit). Код выглядит следующим образом:

```
//+------------------------------------------------------------------+
//|                                              Demo_Adaptive_PSAR.mq5 |
//|                                  Copyright 2026, soloharbinger |
//|                       https://www.mql5.com/ru/users/soloharbinger |
//+------------------------------------------------------------------+
#property copyright "Copyright 2026, soloharbinger"
#property link      "https://www.mql5.com/ru/users/soloharbinger"
#property version   "1.00"

//+------------------------------------------------------------------+
//| Expert initialization function                                   |
//+------------------------------------------------------------------+
int OnInit()
  {
//---

//---
   return(INIT_SUCCEEDED);
  }
//+------------------------------------------------------------------+
//| Expert deinitialization function                                 |
//+------------------------------------------------------------------+
```

```
void OnDeinit(const int reason)
  {
//---

  }
//+------------------------------------------------------------------+
//| Expert tick function                                             |
//+------------------------------------------------------------------+
void OnTick()
  {
//---

  }
```

Код советника начинается с двух директив #include и четырех групп входных параметров. Эти группы входных параметров соответствуют разным задачам внутри системы:

```
#include <Trade/Trade.mqh>
#include <DivergenceEngine.mqh>          // Reusable divergence detection header

//+------------------------------------------------------------------+
//| Input Parameters                                                 |
//+------------------------------------------------------------------+
input group "=== Parabolic SAR Settings ==="
input double   InpSARStep = 0.02;                                   // SAR Step (Accele
input double   InpSARMaximum = 0.2;                                 // SAR Maximum

input group "=== Adaptive Settings ==="
enum ENUM_ADAPTIVE_MODE
  {
   ADAPTIVE_REDUCE,   // Reduce SAR step (slower acceleration)
   ADAPTIVE_INCREASE  // Increase SAR step (faster acceleration)
  };
input bool     EnableAdaptive = false;                              // Enable Adaptive
input ENUM_ADAPTIVE_MODE InpAdaptiveMode = ADAPTIVE_REDUCE;         // Adaptive mode
input double   AdaptiveFactor = 0.3;                                // Adaptive factor
input int      DivBarsLimit = 200;                                  // Max bars since d
input bool     ResetOnTrendChange = false;                          // Reset divergence

input group "=== Divergence Engine Settings ==="
input ENUM_DIV_SOURCE InpSource = DIV_SRC_MPO;                      // Oscillator Sourc
input int    MPO_len = 6;                                           // MPO Length
input int    MPO_smooth = 7;                                        // MPO Smoothing
input int    DivRsiPeriod = 14;                                     // RSI Period
input int    DivPivotLen = 2;                                       // Pivot Length

input group "=== Trade Management ==="
input double RiskPercent = 0.5;                                     // Risk per trade [
input double TPRatio = 2.0;                                         // Take-profit rati
input double FixedLotSize = 0.0;                                    // Fixed lot (0 = u
input int    MagicNumber = 23456;                                   // EA Magic Number
input int    Lookback = 300;                                        // Number of bars f
```

Группа Parabolic SAR Settings задает базовое поведение индикатора. Группа Adaptive Settings задает степень влияния дивергенции на коэффициент ускорения SAR. Группа Divergence Engine Settings содержит настройки, необходимые для инициализации модуля в файле DivergenceEngine.mqh. Группа

Trade Management намеренно выделена отдельно, поскольку нужна только для настройки минимального слоя исполнения торговых операций в демонстрационном примере.

Глобальные переменные

После входных параметров в коде советника определяются переменные, хранящие его состояние во время выполнения:

```
//+------------------------------------------------------------------+
//| Global Variables                                                 |
//+------------------------------------------------------------------+
CTrade trade;
DivergenceEngine engine;                    // Divergence engine instance

double   g_SARCurrent[];                      // SAR values used for current bar (hybrid)
double   openPrices[], highPrices[], lowPrices[], closePrices[];
double   point;
datetime lastBarTime;
int      lastResizeSize = 0;
int      lookback = 0;
```

Наиболее важная переменная здесь — g_SARCurrent[]. Используется только один ряд значений SAR, поскольку в советнике нет отдельных "стандартного" и "адаптивного" индикаторов. Вместо этого выполняется единый расчет значений Parabolic SAR с постоянным учетом текущего адаптивного шага. Такое проектное решение гарантирует, что:

- на графике будет отображаться фактическое значение, используемое для принятия торговых решений;
- состояние индикатора не будет дублироваться;
- адаптация будет осуществляться путем изменения параметра, а не использования второго индикатора.

Другими словами, система не переключается между двумя реализациями SAR, как было бы при использовании двух хэндлов индикатора, полученных с помощью функции iSAR(). Вместо этого система пересчитывает один ряд значений SAR по выбранным в данный момент адаптивным правилам. Один и тот же ряд используется и для отображения, и для исполнения торговых операций.

OnInit() и OnDeinit()

Инициализация выполняется с соблюдением того же принципа разделения ответственности:

```
//+------------------------------------------------------------------+
//| Initialization                                                   |
//+------------------------------------------------------------------+
int OnInit()
  {
   trade.SetExpertMagicNumber(MagicNumber);
   double pointValue = 0.0;
   if(!SymbolInfoDouble(_Symbol, SYMBOL_POINT, pointValue) || pointValue <= 0.0)
     {
      PrintFormat("Failed to get SYMBOL_POINT, error=%d", GetLastError());
      return INIT_FAILED;
     }
   point = pointValue;

//--- Validate inputs
   if(InpSARStep <= 0 || InpSARMaximum <= InpSARStep)
     {
      Print("Invalid PSAR parameters. Step > 0 and Maximum > Step.");
```

```
      return INIT_PARAMETERS_INCORRECT;
     }
   if(AdaptiveFactor < 0 || AdaptiveFactor > 1)
     {
      Print("AdaptiveFactor must be in [0..1]");
      return INIT_PARAMETERS_INCORRECT;
     }
   if(RiskPercent < 0 || TPRatio <= 0)
     {
      Print("RiskPercent must be > = 0 and TPRatio > 0");
      return INIT_PARAMETERS_INCORRECT;
     }

//--- Initialize divergence engine
   if(!InitializeDivergenceEngine(engine, InpSource, MPO_len, MPO_smooth,
                                  DivRsiPeriod, DivPivotLen, _Symbol, PERIOD_CURRENT))
     {
      Print("Failed to initialize Divergence Engine");
      return INIT_FAILED;
     }

//--- Set lookback based on required minimum
   int required = GetRequiredLookback(engine);
   if(Lookback < required)
     {
      Print("Lookback (", Lookback, ") is less than required (", required, "). Using re
      lookback = required;
     }
   else
      lookback = Lookback;

//--- Resize divergence buffers
   if(!ResizeDivergenceBuffers(engine, lookback))
     {
      Print("Failed to resize divergence buffers");
      ReleaseDivergenceEngine(engine);
      return INIT_FAILED;
     }
   lastResizeSize = lookback;

//--- Prepare price arrays
   if(ArrayResize(openPrices, lookback) != lookback ||
      ArrayResize(highPrices, lookback) != lookback ||
      ArrayResize(lowPrices, lookback) != lookback ||
      ArrayResize(closePrices, lookback) != lookback ||
      ArrayResize(g_SARCurrent, lookback) != lookback)
     {
      PrintFormat("ArrayResize for price arrays failed, lookback=%d, error=%d",
                  lookback, GetLastError());
      ReleaseDivergenceEngine(engine);
      return INIT_FAILED;
     }

   if(!ArraySetAsSeries(openPrices, true) ||
      !ArraySetAsSeries(highPrices, true) ||
      !ArraySetAsSeries(lowPrices, true) ||
      !ArraySetAsSeries(closePrices, true) ||
```

```
        !ArraySetAsSeries(g_SARCurrent, true))
      {
       PrintFormat("ArraySetAsSeries for price arrays failed, error=%d", GetLastError());
       ReleaseDivergenceEngine(engine);
       return INIT_FAILED;
      }

//--- Delete old chart objects to start fresh
   int deleted = ObjectsDeleteAll(0, "DynSAR_");
   if(deleted < 0)
      PrintFormat("ObjectsDeleteAll DynSAR_ failed, error=%d", GetLastError());

   deleted = ObjectsDeleteAll(0, "DivArrow_");
   if(deleted < 0)
      PrintFormat("ObjectsDeleteAll DivArrow_ failed, error=%d", GetLastError());

   ChartRedraw(0);
   UpdateMarketData();

   Print("Adaptive PSAR EA initialized.");
   return INIT_SUCCEEDED;
  }
```

Сначала проводится проверка входных параметров. Затем модуль обнаружения дивергенций инициализируется с помощью функции InitializeDivergenceEngine(), после чего советник задает размеры массивов цен и значений SAR, а также буферов модуля. Советник не настраивает модуль вручную. Советник передает необходимые настройки в функцию инициализации, а управление внутренним состоянием и хэндлом индикатора RSI, если он используется, остается в модуле.

В функции OnDeinit() выполняются обратные действия — освобождаются ресурсы:

```
//+------------------------------------------------------------------+
//| Deinitialization                                                 |
//+------------------------------------------------------------------+
void OnDeinit(const int reason)
  {
//--- Delete all visuals and release indicators
   ReleaseDivergenceEngine(engine);
   int deleted = ObjectsDeleteAll(0, "DynSAR_");
   if(deleted < 0)
      PrintFormat("ObjectsDeleteAll DynSAR_ failed, error=%d", GetLastError());

   deleted = ObjectsDeleteAll(0, "DivArrow_");
   if(deleted < 0)
      PrintFormat("ObjectsDeleteAll DivArrow_ failed, error=%d", GetLastError());

   ChartRedraw(0);
  }
```

Функция ReleaseDivergenceEngine() освобождает внутренние ресурсы модуля, а за удаление собственных графических объектов по-прежнему отвечает советник.

OnTick() и единый цикл обработки

Советник выполняет расчеты один раз для каждого бара, а не на каждом тике.

```
//+------------------------------------------------------------------+
//| Main Tick                                                        |
```

```
//+------------------------------------------------------------------+
void OnTick()
  {
//--- New bar check
   datetime currentBarTime = iTime(_Symbol, PERIOD_CURRENT, 0);
   if(currentBarTime == 0 || currentBarTime == lastBarTime)
      return;
   lastBarTime = currentBarTime;

//--- Update prices and SAR/divergence
   if(!UpdateMarketData())
      return;

//--- Check for trade signals and execute
   CheckAndExecuteTrade();
  }
```

После обнаружения нового бара основная обработка выполняется в функции UpdateMarketData(). До вызова торговой логики эта функция получает ценовые данные, обновляет модуль обнаружения дивергенций, рассчитывает значения SAR, запрашивает данные о дивергенциях и обновляет отображение.

Первая часть — простая подготовка ценовых данных:

```
//+------------------------------------------------------------------+
//| Update market data, compute SAR and divergence, draw signals     |
//+------------------------------------------------------------------+
bool UpdateMarketData()
  {
//--- Copy price data
   if(CopyOpen(_Symbol, PERIOD_CURRENT, 0, lookback, openPrices) < lookback ||
      CopyHigh(_Symbol, PERIOD_CURRENT, 0, lookback, highPrices) < lookback ||
      CopyLow(_Symbol, PERIOD_CURRENT, 0, lookback, lowPrices) < lookback ||
      CopyClose(_Symbol, PERIOD_CURRENT, 0, lookback, closePrices) < lookback)
     {
      Print("Failed to copy price data");
      return false;
     }
```

Затем модуль подготавливается к текущему проходу:

```
//--- Update divergence engine (RSI if used)
   if(lookback != lastResizeSize)
     {
      if(!ResizeDivergenceBuffers(engine, lookback))
         return false;
      lastResizeSize = lookback;
     }
   if(!UpdateDivergenceRSI(engine, lookback))
      return false;

   int limit = lookback - MPO_len - DivPivotLen - 3;
   if(limit < 0)
      limit = 0;
   InitDivergenceBoundaries(engine, limit);

//--- Internal Parabolic SAR calculation (backward loop)
```

```
int    trend = closePrices[limit] > openPrices[limit] ? 1 : -1;
double sar   = (trend == 1) ? lowPrices[limit] : highPrices[limit];
double ep    = (trend == 1) ? highPrices[limit] : lowPrices[limit];
double af    = InpSARStep;
```

Эти вызовы важны, поскольку показывают, где именно модуль для повторного использования подключается к существующему советнику. В использующей модуль программе определяется, когда запрашивать данные и какой объем истории обрабатывать, а модуль управляет своими внутренними буферами и расчетами. Значение limit также выбирается так, чтобы между первым обрабатываемым баром и самым старым доступным баром оставалось достаточно истории для расчета осциллятора и обнаружения опорных точек.

Теперь расчет значений SAR и обработка дивергенций выполняются в одном общем цикле с уменьшением индекса:

```
for(int i = limit; i >= 0; i--)
  {
   //--- Calculate oscillator and detect divergence at this bar
   CalculateOscillator(engine, i, openPrices, closePrices, point);
   DetectDivergence(engine, i, highPrices, lowPrices, limit);

   //--- Trend flip detection and reset
   if(trend == 1 && lowPrices[i] < sar)
     {
      trend = -1;
      sar   = ep;
      ep    = lowPrices[i];
      af    = InpSARStep;
      if(ResetOnTrendChange)
         ResetDivergenceState(engine, i);
     }
   else
      if(trend == -1 && highPrices[i] > sar)
        {
         trend = 1;
         sar   = ep;
         ep    = highPrices[i];
         af    = InpSARStep;
         if(ResetOnTrendChange)
            ResetDivergenceState(engine, i);
        }

   //--- Determine adaptive step if divergence opposes current trend
   double currentStep = InpSARStep;
   if(EnableAdaptive)
     {
      int divType   = GetLastDivergenceType(engine, i);
      int barsSince = GetBarsSinceDivergence(engine, i);
      if(barsSince <= DivBarsLimit)
        {
         bool isUptrend = (trend == 1);
         if((isUptrend && divType == -1) || (!isUptrend && divType == 1))
           {
            if(InpAdaptiveMode == ADAPTIVE_REDUCE)
               currentStep = InpSARStep * (1.0 - AdaptiveFactor);  // Apply adaptive
            else // INCREASE
               currentStep = InpSARStep * (1.0 + AdaptiveFactor);  // Apply adaptive
```

```
            // Confine to reasonable bounds
            if(currentStep < 0.001)
               currentStep = 0.001;
            if(currentStep > InpSARMaximum)
               currentStep = InpSARMaximum;
           }
      }
   }
```

Сначала советник вызывает функции модуля обнаружения дивергенций для расчета значения осциллятора и обнаружения дивергенции для текущего бара. И только после этого советник запрашивает у модуля состояние дивергенции. Это гарантирует, что в адаптивной логике SAR будет использоваться состояние, сформированное для этого же бара, а не устаревшие данные.

Блок смены тренда по-прежнему относится к логике SAR. Когда цена пересекает текущее значение SAR, направление тренда меняется, значение ценового экстремума сбрасывается, а коэффициент ускорения возвращается к значению базового шага. Параметр ResetOnTrendChange позволяет дополнительно сбросить состояние предыдущей дивергенции в модуле обнаружения дивергенций, если она уже способствовала развороту тренда.

Затем адаптивный блок считывает данные из модуля через его открытый интерфейс. Советник запрашивает только тип дивергенции и ее давность, определяет, направлена ли она против текущего тренда, и на основе этой информации рассчитывает временное значение currentStep. Другими словами, модуль обнаружения дивергенций не изменяет SAR напрямую. Модуль предоставляет информацию, а логика расчета SAR определяет, как на основе этой информации изменять параметры этого индикатора.

Остальная часть цикла посвящена управлению состоянием SAR:

```
      //--- Update EP and acceleration factor
      if(trend == 1)
        {
         if(highPrices[i] > ep)
           {
            ep = highPrices[i];
            af = MathMin(af + currentStep, InpSARMaximum);
           }
        }
      else
        {
         if(lowPrices[i] < ep)
           {
            ep = lowPrices[i];
            af = MathMin(af + currentStep, InpSARMaximum);
           }
        }

      //--- Store SAR value and draw SAR dot
      g_SARCurrent[i] = sar;
      DrawSARDot(i, g_SARCurrent[i]);

      //--- Draw divergence arrow if divergence just occurred
      if(GetBarsSinceDivergence(engine, i) == 0)
        {
         int divType = GetLastDivergenceType(engine, i);
         if(divType != 0)
            DrawDivergenceArrow(i, divType);
        }
```

```
      //--- Calculate SAR for next bar (i-1)
      if(trend == 1)
        {
         sar = sar + af * (ep - sar);
         sar = MathMin(sar, MathMin(lowPrices[i], (i < limit ? lowPrices[i+1] : lowPric
        }
      else
        {
         sar = sar + af * (ep - sar);
         sar = MathMax(sar, MathMax(highPrices[i], (i < limit ? highPrices[i+1] : highP
        }
     }
   ChartRedraw(0);
   return true;
  }
```

Мы обновили значение ценового экстремума и коэффициент ускорения, сохранили рассчитанное значение SAR, отобразили его на графике, отметили вновь подтвержденные дивергенции и рассчитали значение SAR для следующего бара.

Отображение одной адаптивной точки

Поскольку используется единый расчет значений SAR, на графике отображается только один ряд этих значений:

```
//+------------------------------------------------------------------+
//| Draw a SAR dot on the chart                                      |
//+------------------------------------------------------------------+
void DrawSARDot(int barIndex, double price)
  {
   if(price <= 0)
      return;
   datetime time = iTime(_Symbol, PERIOD_CURRENT, barIndex);
   if(time == 0)
      return;
   string objName = "DynSAR_" + IntegerToString((int)time);
   if(ObjectFind(0, objName) < 0)
     {
      if(!ObjectCreate(0, objName, OBJ_ARROW, 0, time, price))
        {
         PrintFormat("ObjectCreate failed for %s, error=%d", objName, GetLastError());
         return;
        }

      if(!ObjectSetInteger(0, objName, OBJPROP_ARROWCODE, 159))
         PrintFormat("Object ARROWCODE failed for %s, error=%d", objName, GetLastError(
      if(!ObjectSetInteger(0, objName, OBJPROP_COLOR, clrDarkOrange))
         PrintFormat("Object COLOR failed for %s, error=%d", objName, GetLastError());
      if(!ObjectSetInteger(0, objName, OBJPROP_WIDTH, 1))
         PrintFormat("Object WIDTH failed for %s, error=%d", objName, GetLastError());
      if(!ObjectSetInteger(0, objName, OBJPROP_BACK, true))
         PrintFormat("Object BACK failed for %s, error=%d", objName, GetLastError());
      if(!ObjectSetInteger(0, objName, OBJPROP_HIDDEN, true))
         PrintFormat("Object HIDDEN failed for %s, error=%d", objName, GetLastError());
     }
   else
     {
```

```
      if(!ObjectMove(0, objName, 0, time, price))
         PrintFormat("ObjectMove failed for %s, error=%d", objName, GetLastError());
     }
  }
```

Каждая точка идентифицируется исключительно по временной метке соответствующего бара. При первом проходе объект создается, а при последующих перемещается тот же объект. Это означает, что изменение адаптивной настройки меняет положение существующей линии SAR, а не создает вторую "стандартную" или "адаптивную" линию, как это было бы при использовании хэндлов индикаторов. Тем самым советник сравнивает два режима поведения одной и той же реализации.

Стрелки дивергенции отображаются по той же схеме: один объект для каждого бара.

```
//+------------------------------------------------------------------+
//| Draw Divergence Arrow                                            |
//+------------------------------------------------------------------+
void DrawDivergenceArrow(int barIndex, int divType)
  {
   datetime time = iTime(_Symbol, PERIOD_CURRENT, barIndex);
   if(time == 0)
      return;
   string objName = "DivArrow_" + IntegerToString((int)time);
   if(ObjectFind(0, objName) >= 0)
      return; // already exists

   double price;
   int arrowCode;
   color clr;
   if(divType == 1) // Bullish divergence
     {
      price = openPrices[barIndex] - 50 * point;
      arrowCode = 233; // up arrow
      clr = clrDodgerBlue;
     }
   else // Bearish divergence
     {
      price = openPrices[barIndex] + 50 * point;
      arrowCode = 234; // down arrow
      clr = clrCrimson;
     }

   if(!ObjectCreate(0, objName, OBJ_ARROW, 0, time, price))
     {
      PrintFormat("ObjectCreate failed for %s, error=%d", objName, GetLastError());
      return;
     }

   if(!ObjectSetInteger(0, objName, OBJPROP_ARROWCODE, arrowCode))
      PrintFormat("Object ARROWCODE failed for %s, error=%d", objName, GetLastError());
   if(!ObjectSetInteger(0, objName, OBJPROP_COLOR, clr))
      PrintFormat("Object COLOR failed for %s, error=%d", objName, GetLastError());
   if(!ObjectSetInteger(0, objName, OBJPROP_WIDTH, 1))
      PrintFormat("Object WIDTH failed for %s, error=%d", objName, GetLastError());
   if(!ObjectSetInteger(0, objName, OBJPROP_BACK, false))
      PrintFormat("Object BACK failed for %s, error=%d", objName, GetLastError());
   if(!ObjectSetInteger(0, objName, OBJPROP_HIDDEN, true))
```

```
      PrintFormat("Object HIDDEN failed for %s, error=%d", objName, GetLastError());
   }
```

В отличие от точек SAR, маркер дивергенции создается при первом подтверждении дивергенции на данном баре. Благодаря этому график остается наглядным, а положение каждого сигнала сохраняется.

Обнаружение сигналов и исполнение сделок

Торговый слой нужен, чтобы в демонстрационном примере выполнялись торговые операции с измеримыми результатами, а не для решения дополнительной задачи по разработке стратегии. Функция CheckAndExecuteTrade() рассматривает пересечение линии SAR ценой закрытия как сигнал на вход:

```
//+------------------------------------------------------------------+
//| Check for a crossover signal and execute trade                   |
//+------------------------------------------------------------------+
void CheckAndExecuteTrade()
  {
   if(ArraySize(g_SARCurrent) < 3)
      return;

   double sar1 = g_SARCurrent[1];
   double sar2 = g_SARCurrent[2];
   if(sar1 <= 0 || sar2 <= 0)
      return;

//--- Crossover detection (close vs SAR)
   bool buySignal  = (closePrices[2] <= sar2 && closePrices[1] > sar1);
   bool sellSignal = (closePrices[2] >= sar2 && closePrices[1] < sar1);
   if(!buySignal && !sellSignal)
      return;

//--- Check if we already have an open position
   if(PositionsTotal() > 0)
     {
      for(int i = PositionsTotal() - 1; i >= 0; i--)
        {
         ulong ticket = PositionGetTicket(i);
         if(ticket == 0)
           {
            PrintFormat("PositionGetTicket failed at index=%d, error=%d", i, GetLastErr
            return; // state unknown; do not risk duplicate trade
           }

         if(!PositionSelectByTicket(ticket))
           {
            PrintFormat("PositionSelectByTicket failed, ticket=%I64u, error=%d",
                        ticket, GetLastError());
            return;
           }

         if(PositionGetString(POSITION_SYMBOL) == _Symbol &&
            PositionGetInteger(POSITION_MAGIC) == MagicNumber)
            return; // already have our position
        }
     }
//--- Execute trade
   int direction = buySignal ? 1 : -1;
```

```
    ExecuteTrade(direction);
   }
```

В условиях покупки и продажи используется один и тот же ряд значений из массива g_SARCurrent[], отображаемый на графике. Это гарантирует, что торговое решение всегда будет основываться на результатах текущего адаптивного расчета. Перед открытием позиции советник проверяет, нет ли уже открытой позиции по текущему символу со значением MagicNumber.

Затем функция исполнения торговых операций использует значение SAR как ориентир для стоп-лосса и рассчитывает тейк-профит на основе фиксированного соотношения прибыли и риска. Размер позиции может быть фиксированным в лотах или рассчитываться на основе риска:

```
//+------------------------------------------------------------------+
//| Execute trade with SL based on current SAR and TP based on RRR   |
//+------------------------------------------------------------------+
void ExecuteTrade(int direction)
  {
   double price = 0.0;
   if(direction == 1)
     {
      if(!SymbolInfoDouble(_Symbol, SYMBOL_ASK, price) || price <= 0.0)
        {
         PrintFormat("Failed to get the symbol ASK price, error=%d", GetLastError());
         return;
        }
     }
   else
     {
      if(!SymbolInfoDouble(_Symbol, SYMBOL_BID, price) || price <= 0.0)
        {
         PrintFormat("Failed to get the symbol BID price, error=%d", GetLastError());
         return;
        }
     }
   double sar = g_SARCurrent[1]; // SAR at bar 1 (entry bar)
   if(sar <= 0)
      return;

   double stopDist = MathAbs(price - sar);
   long digitsValue = 0;
   if(!SymbolInfoInteger(_Symbol, SYMBOL_DIGITS, digitsValue))
     {
      PrintFormat("Failed to get symbol digits, error=%d", GetLastError());
      return;
     }
   int digits = (int)digitsValue;

   long stopLevelValue = 0;
   if(!SymbolInfoInteger(_Symbol, SYMBOL_TRADE_STOPS_LEVEL, stopLevelValue))
     {
      PrintFormat("Failed to get SYMBOL_TRADE_STOPS_LEVEL, error=%d", GetLastError());
      return;
     }
   int stopLevel = (int)stopLevelValue;

   if(point <= 0)
      return;
```

```
   double sl = NormalizeDouble(sar, digits);
   double minDist = stopLevel * point;
   if(stopDist < minDist)
     {
      stopDist = minDist * 1.5;
      // Recalculate SL based on new distance
      if(direction == 1)
         sl = NormalizeDouble(price - stopDist, digits);
      else
         sl = NormalizeDouble(price + stopDist, digits);
     }

   double tp = (direction == 1) ? price + stopDist * TPRatio : price - stopDist * TPRat
   tp = NormalizeDouble(tp, digits);

//--- Lot size: either fixed or risk-based
   double lotSize = FixedLotSize;
   if(lotSize <= 0 && RiskPercent > 0)
     {
      ResetLastError();
      double balance = AccountInfoDouble(ACCOUNT_BALANCE);
      if(GetLastError() != 0)
        {
         PrintFormat("Failed to get ACCOUNT_BALANCE, error=%d", GetLastError());
         return;
        }

      if(balance <= 0.0)
        {
         Print("Account balance is zero or negative");
         return;
        }

      double tickSize = 0.0, tickValue = 0.0;
      double minLot = 0.0, maxLot = 0.0, stepLot = 0.0;
      if(!SymbolInfoDouble(_Symbol, SYMBOL_TRADE_TICK_SIZE, tickSize) ||
         !SymbolInfoDouble(_Symbol, SYMBOL_TRADE_TICK_VALUE, tickValue) ||
         !SymbolInfoDouble(_Symbol, SYMBOL_VOLUME_MIN, minLot) ||
         !SymbolInfoDouble(_Symbol, SYMBOL_VOLUME_MAX, maxLot) ||
         !SymbolInfoDouble(_Symbol, SYMBOL_VOLUME_STEP, stepLot))
        {
         PrintFormat("Failed to read symbol trade properties, error=%d", GetLastError()
         return;
        }

      double riskAmount = balance * RiskPercent / 100.0;
      if(tickValue <= 0 || tickSize <= 0)
         return;
      // Risk per lot in account currency
      double moneyPerPriceUnit = tickValue / tickSize;
      double monetaryRiskPerLot = stopDist * moneyPerPriceUnit;
      if(monetaryRiskPerLot <= 0)
         return;
      lotSize = riskAmount / monetaryRiskPerLot;
      if(stepLot <= 0)
         return;
      lotSize = MathMax(minLot, MathMin(lotSize, maxLot));
```

```
        lotSize = NormalizeDouble(lotSize / stepLot, 0) * stepLot;
      }
   if(lotSize <= 0)
      return;

//--- Open trade
   bool success = false;
   if(direction == 1)
      success = trade.Buy(lotSize, _Symbol, 0, sl, tp, "Adaptive PSAR Buy");
   else
      success = trade.Sell(lotSize, _Symbol, 0, sl, tp, "Adaptive PSAR Sell");

   if(success)
      Print("Trade opened: ", (direction==1?"Buy":"Sell"), " lot=", lotSize, " SL=", sl
   else
      PrintFormat("Trade failed: retcode=%u, description=%s, error=%d",
                  trade.ResultRetcode(),
                  trade.ResultRetcodeDescription(),
                  GetLastError());
  }
```

На этом создание демонстрационного советника завершено.

Таким образом, в демонстрационном примере задачи распределены следующим образом: советник отвечает за рыночные данные, расчет значений SAR, правила адаптации, отображение и исполнение торговых операций, а модуль в файле DivergenceEngine.mqh — за обнаружение дивергенций и управление их состоянием. Взаимодействие между ними ограничивается инициализацией, обновлением буферов, расчетами для каждого бара и небольшим набором открытых функций доступа. Именно такое разграничение задач — основной результат, который этот пример призван продемонстрировать.

Результаты

Советник тестируется в два этапа. Сначала проводится сравнение на двух расположенных рядом графиках в реальном времени, чтобы наглядно подтвердить изменение поведения индикатора под действием механизма адаптации. Затем выполняется сравнение в Тестере стратегий, чтобы проверить, влияет ли это изменение на результаты торговли за определенный исторический период. На обоих этапах изменяются параметры адаптации и связанные с ними входные параметры, а все остальное остается неизменным.

Сравнение на двух графиках

Демонстрационный советник был подключен к двум расположенным рядом графикам одного и того же символа и таймфрейма. Изначально в обоих экземплярах советника использовались настройки по умолчанию.

График 1 находится слева, а график 2 — справа.

При настройках адаптации и всех остальных параметрах по умолчанию на обоих графиках линии SAR, рассчитанные внутри советника, совпадают. Это свидетельствует о воспроизводимости расчета: при одинаковых входных параметрах на одних и тех же барах получаются одинаковые точки. Поэтому любые различия после изменения входных параметров советника на графике 1 обусловлены исключительно настройками адаптации. Из этого не следует, что линия, рассчитанная в коде советника, совпадает с линией встроенного индикатора Parabolic SAR. Цикл расчета SAR в советнике начинается с самого старого бара его собственного расчетного окна, а не с начала истории графика, поэтому не гарантируется, что его точки будут совпадать с точками встроенного индикатора.

Затем были изменены входные параметры советника на графике 1: параметру EnableAdaptive было присвоено значение true, а для адаптации был выбран режим уменьшения коэффициента ускорения с коэффициентом адаптации 0,7.

Настройки советника на графике 2 оставили без изменений, и этот график служил неизменным ориентиром для дальнейшего сравнения. Вот результат:

После появления стрелок дивергенции следует обратить внимание на расстояние точек SAR от цены: на графике 1 оно больше, чем у соответствующих точек на графике 2. Это соответствует выбранному режиму адаптации.

Сравнение эквити

На втором этапе используется Тестер стратегий, чтобы проверить, влияет ли адаптация на основе дивергенций на результаты торговли, а не только на отображение индикатора. Оба теста проводятся на одном и том же символе и таймфрейме, за один и тот же короткий период (с 1 по 15 августа), с одинаковым депозитом и одинаковыми входными параметрами управления торговлей. Различаться будут только параметры группы Adaptive. Сначала проверяем эквити при настройках по умолчанию.

Далее включаем адаптацию и выбираем режим увеличения коэффициента ускорения с коэффициентом адаптации 0,7.

Остальные входные параметры и настройки оставляем без изменений. Получаем следующий результат по эквити:

> Результат по эквити, приведенный в этой статье, предназначен для иллюстрации поведения советника, а не для демонстрации статистического преимущества.

Изменение настроек адаптации привело к другим результатам в Тестере стратегий за период тестирования. Тот же модуль обнаружения дивергенций, который адаптировал множитель ATR индикатора SuperTrend во второй части серии, теперь выполняет ту же задачу для другого индикатора.

Возможности

Применение подключаемого файла DivergenceEngine.mqh и схемы подключения модуля не ограничивается отображением точек Parabolic SAR. Ширину полос Боллинджера, порог гистограммы MACD, период скользящей средней, фильтр ADX или шаг сетки советника можно связать с одним и тем же состоянием дивергенции, поскольку их параметры допускают адаптацию. Применение модуля не ограничивается одним советником или индикатором.

Некоторые индикаторы и советники также могут служить инструментами для других советников или индикаторов. В этом демонстрационном примере Adaptive Parabolic SAR использовался для определения моментов входа в позицию. Однако в большинстве систем Parabolic SAR применяется для переноса стоп-лосса при сопровождении позиции, а не для формирования сигнала на вход. Тот же адаптивный коэффициент ускорения можно использовать как инструмент для ужесточения или ослабления трейлинг-стопа по уже открытой любым другим советником позиции.

Заключение

В этой статье рассматривалось конкретное ограничение: логика обнаружения дивергенций, разработанная в частях I-III, работала, но оставалась привязанной к SuperTrend. Выделение этой логики в файл DivergenceEngine.mqh позволило создать модуль для повторного использования. Его можно подключить к любому индикатору или советнику с помощью единого набора вызовов настройки, обновления, расчета, получения данных и освобождения ресурсов.

Пример с Parabolic SAR продемонстрировал это разделение на практике. Для использования с другим индикатором модуль не потребовал изменений: он по-прежнему предоставлял состояние дивергенции и ее давность, а сглаженное значение осциллятора оставалось доступно через тот же слой доступа. Трактовка и применение этой информации определялись в советнике. Пример с коэффициентом ускорения также показал, что адаптация может как увеличивать, так и уменьшать параметр в зависимости от того, как устроена использующая модуль программа.

Исходные файлы, использованные в этой статье, прикреплены ниже.

Имя файла	Описание
DivergenceEngine.mqh	Подключаемый файл с кодом на языке MQL5 для повторного использования в индикаторах и советниках. В нем объединены расчет значений осциллятора, обнаружение опорных точек, классификация дивергенций и управление их состоянием.
Demo_Adaptive_PSAR.mq5	Простой демонстрационный советник, который объединяет модуль из файла DivergenceEngine.mqh с расчетом значений Parabolic SAR непосредственно в коде советника и изменяет коэффициент ускорения этого индикатора при обнаружении дивергенции, направленной против текущего тренда.

Следующая статья >>

Solomon Anietie Sunday

Manager, Trader and Developer в Self Employed Нигерия 4044

I am an experienced MQL5 developer, Author, Prop Trader, young CEO, specializing in the design, development, and optimization of Expert Advisors (EAs), custom indicators, and trading utilities/tools for MetaTrader 5. Specialized in Trend-based, Indicator based, adaptive, risk management, clean, and well-documented code/strategies.
I value clear communication, learning, well thought out logic implementation, and stable results in real trading conditions.

Другие статьи автора

- Построение системы обнаружения дивергенций (Часть III): Советник Adaptive SuperTrend
- Построение системы обнаружения дивергенций (Часть II): Пользовательский индикатор Adaptive SuperTrend
- Построение системы обнаружения дивергенций: Создание пользовательского индикатора MPO4
- Использование Экономического календаря MQL5 для фильтрации новостей (Часть 4): Точное тестирование на истории со статическими данными
- Использование Экономического календаря MQL5 для фильтрации новостей (Часть 3): Сохранение работы при перезапусках терминала во время новостного окна
- Использование Экономического календаря MQL5 для фильтрации новостей (Часть 2): Временное удаление и восстановление стоп-уровней во время выхода новостей

Подпишитесь и узнавайте о
популярных статьях и кодах